

What Crosses the Substrate-Cell Boundary: Permitted Crossings, Forbidden Crossings, and Provenance Generation in the Coordination Knowledge Substrate Pattern

A derivation note from the Coordination Knowledge Substrate (CKS) pattern.

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: 2 May 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new commitments beyond those defended in the source paper. Its sole contribution is to articulate, in operational form, what crosses the substrate-cell boundary the source paper draws — the classes of operations the architecture permits, the classes it forbids, and the provenance generation that crossings produce as architectural side effect — so downstream work can identify boundary-coherent and boundary-violating systems without ambiguity. This note is the third of five operational-decomposition notes paired with the foundational substrate-cell-boundary note A1.02: companion note A2.08 (substrate-side standalone), companion note A2.09 (cell-side standalone), this note (boundary crossings), planned note A2.11 (cell-internal vs. substrate state), and planned note A2.12 (cell-to-cell communication through substrate).

Abstract

The Coordination Knowledge Substrate (CKS) pattern draws an architectural boundary between substrate (the persistent, human-governed coordination artifact) and cell (the unit of execution that operates over substrate content under human-authored orchestration rules). Companion notes A2.08 and A2.09 formalize what each layer commits to in isolation. This note formalizes what happens at the boundary itself: three classes of permitted crossings (substrate→cell reads, cell→substrate writes, substrate-mediated cell-to-cell flows), four classes of forbidden crossings (cell-internal state escape, authority claims upward, ungoverned writes, direct cell-to-cell flows), provenance generation as architectural content of writes, seven failure modes, and an operational test for whether a system's crossing operations are CKS-coherent.

1. Why the boundary's crossing operations need standalone formalization

The parent foundational note A1.02 commits CKS to a substrate-cell boundary as a joint architectural property. Companion notes A2.08 and A2.09 formalize each layer's standalone identity — A2.08 names what the substrate layer commits to alone, A2.09 names what the cell layer commits to alone.

Layer-level commitments do not exhaust the boundary. A system can satisfy A2.08's four substrate commitments and A2.09's four cell commitments at each layer in isolation while still failing

CKS-coherence at the boundary, because crossings happen in ways the architecture does not permit — direct cell-to-cell flows that bypass the substrate, "fast path" writes that skip rule evaluation, reads that leak cell-internal state, bidirectional flows that erase the asymmetry between reads and writes. Each preserves layer-internal compliance while violating the boundary.

Provenance is the second motivation. The path-retraceability commitment of companion note A1.07 requires that every piece of substrate content carry six fields of provenance metadata, generated when content comes into being. The boundary is where that generation happens — specifically at cell→substrate writes. If crossings do not produce provenance, retraceability fails downstream regardless of either layer's commitments. Naming the boundary's crossing operations as standalone is what makes provenance generation an architectural commitment of the boundary itself rather than an afterthought.

2. The three classes of permitted crossings

The architecture permits three and only three classes of operations to cross the substrate-cell boundary.

Class 1 — Substrate→cell reads. A cell reads substrate content as input to its execution. The read has four operational properties: (a) the cell receives substrate content in *inspectable* form — the same form humans inspect under the inspect right (companion note A2.01) — though embedded representations or other transformed forms may be derived for the cell's internal use; (b) the read accesses content within the cell's orchestration-rule-defined scope (per A2.09's bounded-scope commitment); (c) the read does not modify substrate content as a side effect — reads that update access logs, increment view counters, or append observation events are conflating reads with writes, and those state changes belong architecturally to a separate Class-2 operation; (d) when the read is causally connected to a downstream cell→substrate write, it is recorded in the antecedent-reference field of that write's provenance (per A1.07).

Class 2 — Cell→substrate writes. A cell writes its outputs to the substrate, where they take effect as substrate state attributed to the cell's execution. The write has five operational properties: (a) the write is authorized by orchestration rules at the moment it occurs — authorization is a precondition, not a downstream check; (b) the write produces content shaped by the substrate's schema, not arbitrary state; (c) the write carries the six provenance fields of A1.07 — writer attribution, timestamp, antecedent reference, rule reference, rationale (where the rule requires it), and relationship to contradicting content — and the provenance is itself substrate content; (d) the write takes effect at commit time, with no "pending" or "draft" state requiring external approval — rule evaluation gates the write; writes that pass rule evaluation commit, and writes that fail are not attempted; (e) the write preserves contradictions (per Claim 3, source paper §3, §5): when the cell's output contradicts existing substrate content, the cell records the new content with explicit relationship to the contradicting content rather than silently overwriting it.

Class 3 — Substrate-mediated cell-to-cell flows. When cell A's output influences cell B's behavior, the influence flows through the substrate: cell A performs a Class-2 write, cell B performs a Class-1 read of the resulting substrate content, and cell B responds to the substrate state. The architectural commitment is that no direct flow between cells exists outside this pattern — no shared memory, message queues, callback registrations, or other inter-cell mechanism. The full standalone treatment is the topic of planned note A2.12.

The three classes are exhaustive: every operation that legitimately crosses the boundary belongs to one of them. A specific implementation may use any technical pattern — function calls, REST or GraphQL APIs, file-system operations, database queries, message buses, LLM API calls — to realize the architectural crossings. The architecture commits to the class structure, not to the technical mechanism; tool-agnosticism (companion note A1.05) governs the implementation question. When a cell's read or write is realized through an LLM operating as substrate mediator (per companion note A1.04), the crossing class is unchanged: the LLM mediates the operation under cell scope and orchestration rules, and the read remains a Class-1 read while the write remains a Class-2 write.

3. The four classes of forbidden crossings

The architecture forbids four classes of operations from crossing the substrate-cell boundary. The four are not implementation defects but architectural violations that mark the system as no longer CKS-coherent on the boundary axis.

Forbidden 1 — Cell-internal state escaping to substrate without an explicit write. A cell's internal execution state — intermediate reasoning, scratchpad content, LLM context windows, draft outputs that did not complete the rule's logic — is bounded to the execution. When such state becomes substrate content without an explicit cell→substrate write under rules, the substrate now carries content that circulates without provenance, without rule authorization, and without contradiction-preservation discipline. The full standalone treatment is planned note A2.11.

Forbidden 2 — Authority claims flowing upward from cell to substrate. Cells operate under rules; they do not have governance authority over substrate content or over the rules themselves. When a cell→substrate write attempts to override existing content outside what rules authorize, modify or rewrite the rules, or bypass conflict-preservation, the cell is claiming authority it does not have. Authority over substrate content and rules sits with humans (per A1.01); the boundary is violated by what the write attempts, not only by what gets committed.

Forbidden 3 — Ungoverned writes. When cell→substrate writes occur outside any orchestration rule's authorization — writes that no active rule permits, writes triggered by emergent agent behavior outside rule evaluation, writes from "shortcut" code paths that bypass rule evaluation for performance or convenience — the rule-governed-behavior commitment of A2.09 is violated. The test is not whether a rule could in principle have authorized the write; the test is whether a rule did authorize it at the moment the write occurred.

Forbidden 4 — Direct cell-to-cell flows outside the substrate. When state flows from cell A to cell B without going through the substrate — through shared memory regions, direct API calls between cells, message queues carrying coordination state, callback registrations, side-channel signaling — the substrate-mediation commitment is violated. This is the most consequential forbidden class for the source-of-truth commitment formalized in companion note A1.08, because cell-to-cell state that bypasses the substrate is invisible to the substrate's authoritative record. The substrate may continue to exist; it is no longer authoritative.

4. Provenance generation as architectural content of crossings

The path-retraceability commitment of A1.07 names six fields of provenance metadata that every piece of substrate content must carry. The boundary is the architectural locus where those fields are generated

— at every cell→substrate write, as part of the write operation.

Provenance generation is not separable from the write. A boundary write that does not produce provenance is not a CKS-coherent write, even when it is technically authorized and produces substrate-shaped content. Substrate content cannot be retraced unless the crossing that produced it produced its trace, and retraceability depends on every write having generated provenance at the moment it crossed the boundary. Implementations that defer provenance generation to a later step or treat it as a downstream observability concern have decoupled provenance from the write, and the resulting substrate content has gaps that cannot be filled retrospectively.

Substrate→cell reads contribute to provenance indirectly: the antecedent-reference field of a cell→substrate write records which substrate content the cell read as input. This makes Class-1 reads traceable through the Class-2 writes they led to, without requiring reads to generate their own provenance records. Provenance is itself substrate content, governed by the same human-governed authority commitments (A1.01) and source-of-truth commitments (A1.08) as any other substrate content.

5. Failure modes that violate the boundary's crossing semantics

Each anti-pattern below names a way an implementation can violate the crossing classes specified in §§2–3. Each maps to one of the four forbidden classes or to a property violation of Class-1 or Class-2.

(a) Provenance-missing writes. Cell→substrate writes occur without recording the six provenance fields. The write enters substrate state but cannot be traced to its origin. *Violates Class-2 property (c).*

(b) Side-effect reads. Substrate→cell reads modify substrate content as a side effect — access logs that themselves become substrate content, view-count increments, observation events triggered on read. *Violates Class-1 property (c).*

(c) Pending writes. Cell→substrate writes produce pending or draft state requiring external approval to become effective. Writes that pass rule evaluation should commit; writes that do not pass should not have been attempted. *Violates Class-2 property (d).*

(d) Cell-internal state leakage. Cell-internal state — LLM context, intermediate reasoning, scratchpad content — appears in substrate content without an explicit cell→substrate write under rules. *Violates Forbidden 1.*

(e) Authority-claiming writes. Cell→substrate writes attempt to override existing content outside rule authorization, modify rules, or bypass conflict preservation. *Violates Forbidden 2.* The boundary is violated by the write's intent, not only by its content.

(f) Ungoverned fast paths. Code paths bypass rule evaluation for performance or convenience, producing cell→substrate writes that no active rule authorized. *Violates Forbidden 3.*

(g) Direct cell-to-cell channels. State flows from cell A to cell B through mechanisms that bypass the substrate — shared memory, direct API calls, message queues carrying coordination state, callback registrations. *Violates Forbidden 4.*

A further failure mode worth naming is **symmetric boundary treatment**: the architecture treats reads and writes as equivalent inverse operations, losing the asymmetric governance content of the two

directions. Reads grant access under scope rules; writes commit state under authorization rules and produce provenance. Collapsing the two into a generic R/W interface dissolves the architectural distinction between Class-1 and Class-2 at the implementation layer, with consequences that propagate to provenance, authority, and source-of-truth commitments.

6. Operational test

A system's substrate-cell boundary crossings are CKS-coherent if and only if the following are all true at all times during the substrate's existence:

1. Substrate→cell reads return substrate content in inspectable form, within the cell's bounded scope, without modifying substrate content as a side effect.
2. Cell→substrate writes are authorized by orchestration rules at the moment of the write, produce substrate-shaped content, carry the six provenance fields, take effect as substrate state at commit time, and preserve contradictions rather than silently overwriting contradicting content.
3. Cell-to-cell flows occur exclusively through the substrate (cell A writes; cell B reads), with no direct channels between cells.
4. Cell-internal execution state does not become substrate content without an explicit cell→substrate write under orchestration rules.
5. Cells do not claim governance authority over substrate content or over orchestration rules; writes that exceed rule authorization are architecturally rejected, not committed.
6. Provenance is generated at every cell→substrate write as part of the write operation, not as an adjacent, optional, or deferred feature.
7. Reads and writes are architecturally distinct operation classes, with their respective governance content (scope rules for reads; authorization rules and provenance generation for writes) preserved rather than collapsed into a symmetric R/W interface.

A system that fails any of (1)–(7) is not CKS-coherent on the boundary-crossing axis, even where its substrate layer (per A2.08) and cell layer (per A2.09) are individually compliant. The boundary is an architectural object in its own right; layer compliance does not entail boundary compliance.

7. Conclusion

The substrate-cell boundary is not a residual of the two layers and not a generic interface between them. It is an architectural object with its own operational content: three classes of permitted crossings, four classes of forbidden crossings, and provenance generation that occurs at writes as part of the write operation. Implementations that treat the boundary as a wire-protocol channel, a function-call interface, an API contract, a message-bus topology, or an event-sourcing flow may realize permitted crossings using those mechanisms, but the architectural content lives in the class structure of crossings, not in the technical mechanism that realizes them.

Two consequences follow from naming the boundary's content as standalone. First, governance failures are identifiable as architectural violations rather than as integration bugs: a cell→substrate write that bypasses rule evaluation is not a missed validation step but a violation of Forbidden 3. Second, traceability failures are identifiable at their source: substrate content that lacks provenance was produced by a non-coherent write, and the remediation is at the boundary write rather than at

downstream observability infrastructure.

With this note complete, paired with companion notes A2.08 and A2.09, the operational decomposition of the substrate-cell boundary stated in foundational note A1.02 has its three load-bearing pieces formalized: the substrate side commits to four properties; the cell side commits to four properties; the boundary itself permits three classes, forbids four, and generates provenance at writes. Planned notes A2.11 and A2.12 develop two specific aspects of the boundary in further detail. Subsequent work that uses "boundary crossing" differently — that conflates crossings with API operations, treats the boundary as symmetric, or treats provenance generation as adjacent to writes rather than constitutive of them — is using a different concept, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *What Crosses the Substrate-Cell Boundary: Permitted Crossings, Forbidden Crossings, and Provenance Generation in the Coordination Knowledge Substrate Pattern*. 2 May 2026. ORCID: 0009-0004-8065-3235.